CUSTOMER SHOPPING BEHAVIOR ANALYSIS

Updated End-to-End Data Analytics Project Report

Python → MySQL → SQL → Power BI Dashboard

Component	Evidence included
Python	Actual notebook outputs from the uploaded .ipynb
SQL	Actual uploaded SQL queries + clearly separated dashboard result evidence
Power BI	Actual dashboard screenshot supplied by the user
Report	Business insights, recommendations and portfolio structure

Important: The uploaded SQL file contains queries but does not contain SQL-console result tables. Therefore this report does not invent SQL output values. Where possible, dashboard results are shown as dashboard evidence.

1. Executive Summary

This updated report documents the actual workflow present in the uploaded project files: customer shopping data is inspected and cleaned in Python, transformed into an analysis-ready DataFrame, loaded into MySQL, analyzed using SQL, and presented in Power BI.

Metric	Actual evidence
Rows in raw dataset	3,900
Original columns	18
Missing Review Rating values before cleaning	37
Missing values after cleaning	0
Average purchase amount	59.764359 (≈ 59.76)
Average review rating	3.750065 (≈ 3.75)
Power BI customer KPI	4K
Power BI subscription mix	Yes 27% / No 73%

2. Python — Dataset Loading & Actual Output

The notebook loads the customer_shopping_behavior.csv file with Pandas and immediately displays df.head(). The output below is reproduced from the uploaded notebook.

Code

```
df = pd.read_csv('customer_shopping_behavior.csv')
df.head()
```

Actual notebook output

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount
1	55	Male	Blouse	Clothing	53
2	19	Male	Sweater	Clothing	64
3	50	Male	Jeans	Clothing	73
4	21	Male	Sandals	Footwear	90
5	45	Male	Blouse	Clothing	49

The notebook output also shows the remaining original fields for these rows, including location, size, color, season, review rating, subscription status, shipping type, discount/promo fields, previous purchases, payment method and purchase frequency.

3. Python — Data Profiling Output

The uploaded notebook's df.info() confirms 3,900 records and 18 columns. Review Rating is the only column with missing values at this stage.

Column / measure	Actual notebook output
DataFrame rows	3,900
Total columns	18
Review Rating non-null	3,863
Review Rating missing	37
Age dtype	int64
Purchase Amount dtype	int64
Review Rating dtype	float64
Object columns	13

Descriptive statistics from the notebook

Measure	Age	Purchase Amount	Review Rating
Count	3900	3900	3863
Mean	44.068462	59.764359	3.750065
Min	18	20	2.5
25%	31	39	3.1
50%	44	60	3.8
75%	57	81	4.4
Max	70	100	5.0

4. Python — Cleaning & Transformation Outputs

The notebook fills missing Review Rating values using the median within each Category. The second null-count output shows zero missing values across all columns.

Stage	Actual output
Before cleaning — Review Rating	37 missing
After category-median fill — Review Rating	0 missing
All other fields after cleaning	0 missing

Column standardization

The notebook converts column names to lowercase, replaces spaces with underscores and renames purchase_amount_(usd) to purchase_amount.

Age-group output

age	age_group
55	Middle-aged
19	Young Adult
50	Middle-aged
21	Young Adult
45	Middle-aged
46	Middle-aged
63	Senior
27	Young Adult
26	Young Adult
57	Middle-aged

5. Python — Feature Engineering & Validation Output

Purchase frequency is converted into a numeric day field using the mapping defined in the notebook.

frequency_of_purchases	purchase_frequency_days
Fortnightly	14
Weekly	7
Weekly	7
Weekly	7
Annually	365
Weekly	7
Quarterly	90
Weekly	7
Annually	365
Quarterly	90

Redundant-field validation

The notebook compares discount_applied with promo_code_used and returns **np.True_**, confirming that the two fields match for all records. The notebook then drops promo_code_used.

Final prepared schema

customer_id, age, gender, item_purchased, category, purchase_amount, location, size, color, season, review_rating, subscription_status, shipping_type, discount_applied, previous_purchases, payment_method, frequency_of_purchases, age_group, purchase_frequency_days

6. Python → MySQL Connection — Actual Output

The notebook creates the customer_behavior database if needed, connects to MySQL through SQLAlchemy, and writes the prepared DataFrame to the customer table.

Actual notebook output

Output line	Actual value
Load status	Data successfully loaded into MySQL!
Database	customer_behavior
Table	customer

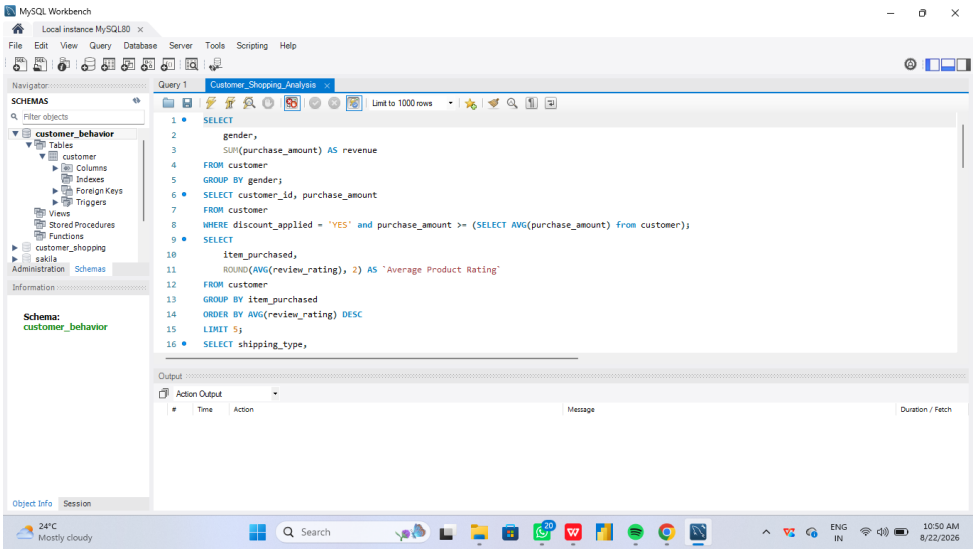
Portfolio security note

Before publishing the notebook to GitHub, remove the hard-coded local database password from the connection string and use environment variables or a local configuration file excluded from Git.

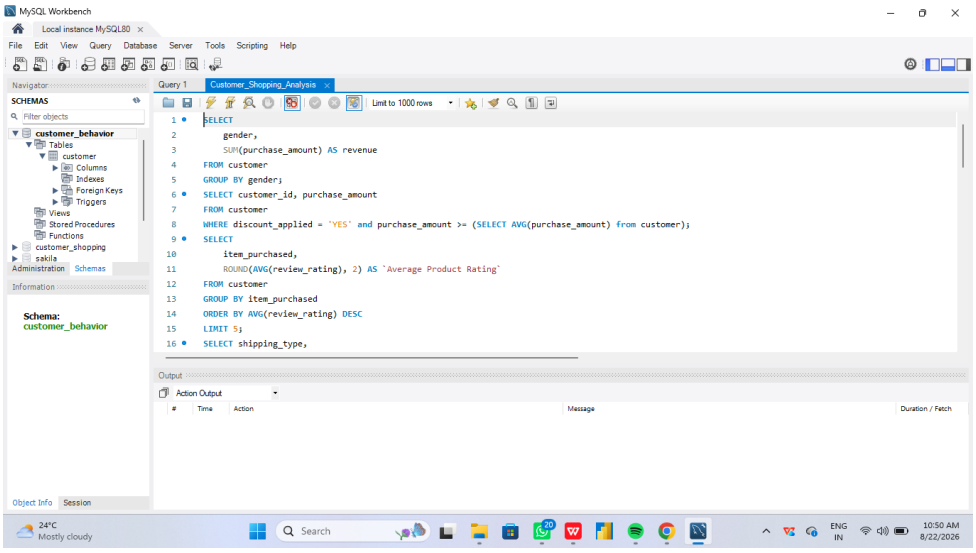
SQL Query Evidence — MySQL Workbench

This section has been added to the previous project report so the SQL work is supported by actual MySQL Workbench screenshots. The screenshots below show the query-development stage used for the Customer Shopping Behavior analysis.

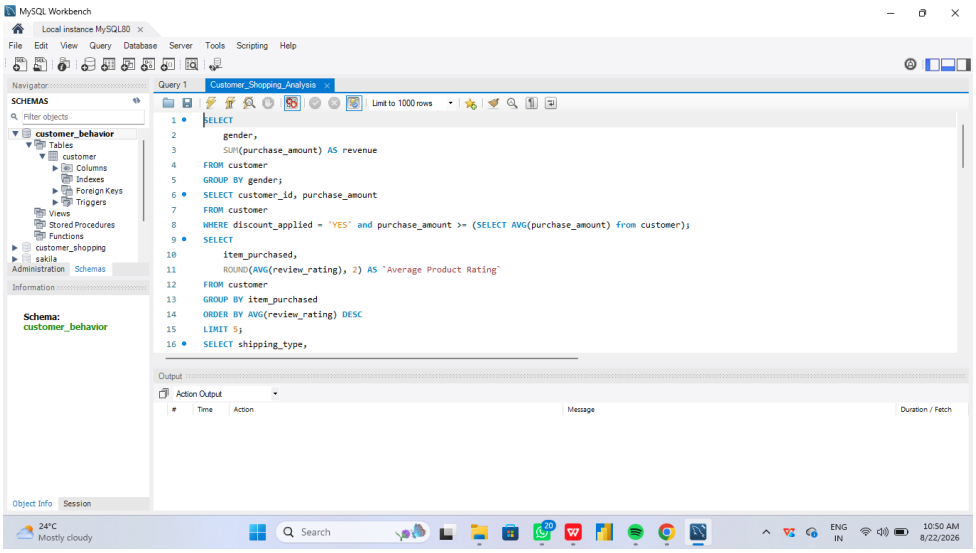
Query development screenshots



Screenshot 1 — SQL queries prepared in MySQL Workbench.



Screenshot 2 — Continued SQL analysis queries.

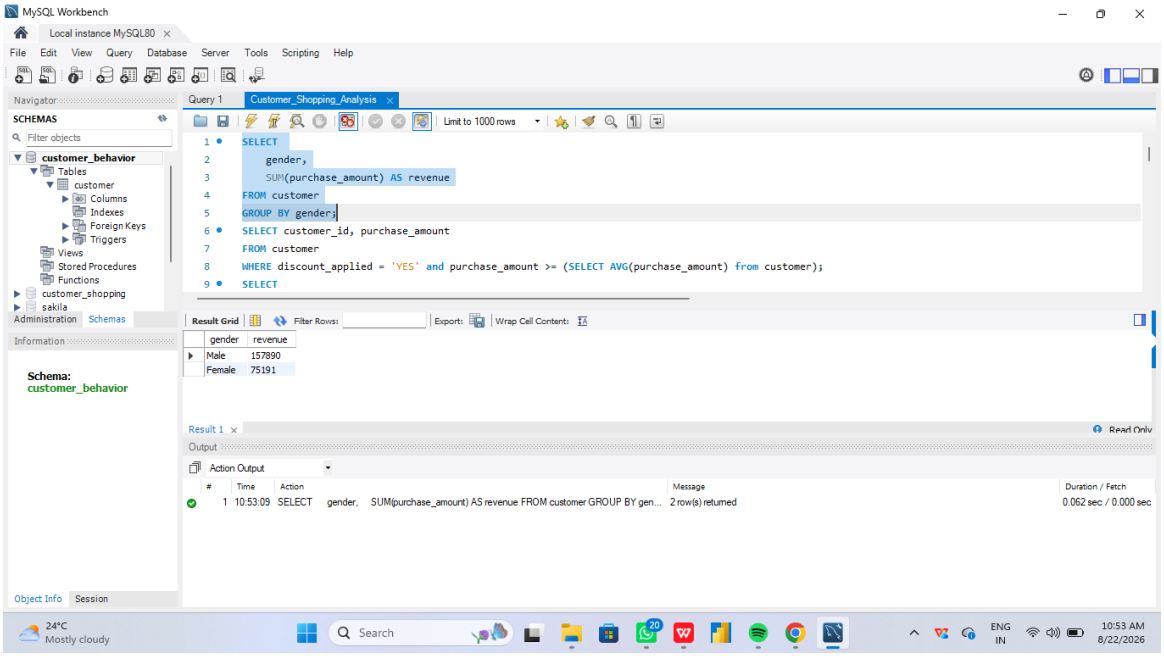


Screenshot 3 — Query set including product-rating analysis.

SQL Output Evidence — Actual Results

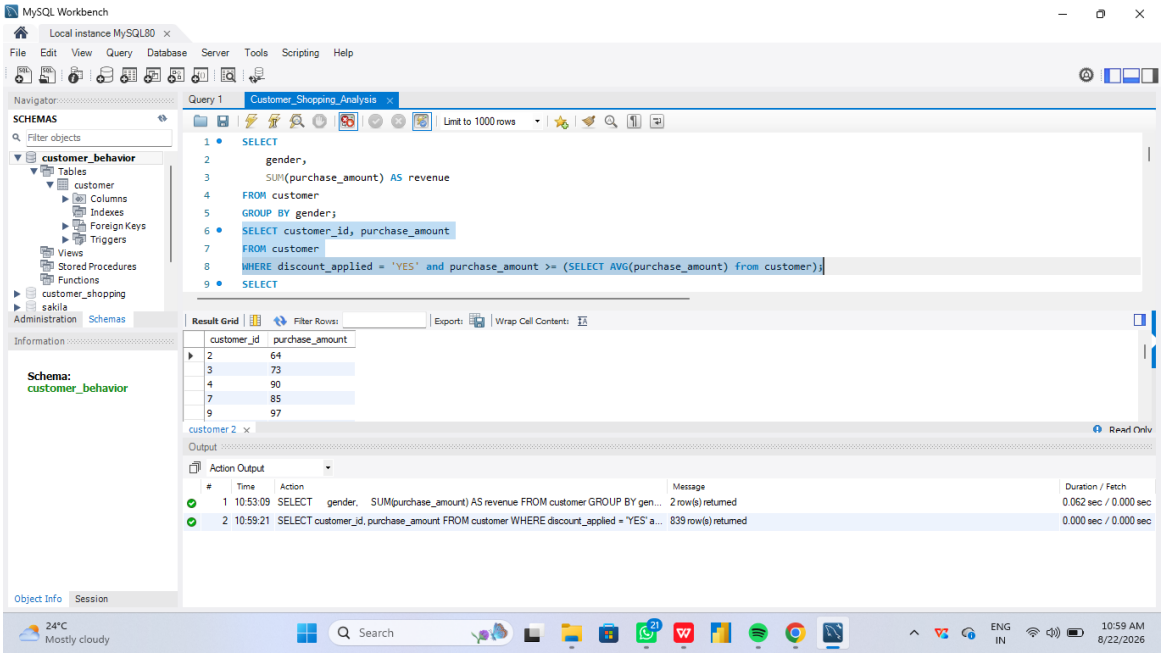
The following screenshots are the important SQL result evidence that was missing from the earlier report. They show MySQL Workbench executing the queries and returning actual result rows.

1. Revenue by Gender — SQL output



Visible result: Male revenue = 157,890 and Female revenue = 75,191. The query groups customer records by gender and calculates SUM(purchase_amount).

2. Discount-applied customers above average purchase amount — SQL output



Visible result: the query returned 839 rows. The screenshot also shows sample returned records including customer IDs 2, 3, 4, 7 and 9 with purchase amounts 64, 73, 90, 85 and 97.

7. SQL Analysis — Queries & Result Evidence

The uploaded SQL file contains the following business analyses. The source file contains the SQL statements but no captured result tables. To keep this report accurate, the query text is shown exactly as project evidence, while dashboard values are labeled separately as Power BI evidence.

Revenue by gender

```
SELECT gender, SUM(purchase_amount) AS revenue FROM customer GROUP BY gender;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Discounted purchases above average

```
SELECT customer_id, purchase_amount FROM customer WHERE discount_applied = 'YES' AND purchase_amount >= (SELECT
AVG(purchase_amount) FROM customer);
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Top 5 products by average rating

```
SELECT item_purchased, ROUND(AVG(review_rating), 2) AS `Average Product Rating` FROM customer GROUP BY
item_purchased ORDER BY AVG(review_rating) DESC LIMIT 5;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Average spend by shipping type

```
SELECT shipping_type, ROUND(AVG(purchase_amount), 2) FROM customer WHERE shipping_type IN
('Standard','Express') GROUP BY shipping_type;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Subscription analysis

```
SELECT subscription_status, COUNT(customer_id) AS total_customers, ROUND(AVG(purchase_amount),2) AS avg_spend,
ROUND(SUM(purchase_amount),2) AS total_revenue FROM customer GROUP BY subscription_status;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Top 5 products by discount rate

```
SELECT item_purchased, ROUND(SUM(CASE WHEN discount_applied='YES' THEN 1 ELSE 0 END)/COUNT(*)*100,2) AS
discount_rate FROM customer GROUP BY item_purchased ORDER BY discount_rate DESC LIMIT 5;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Customer segmentation

```
WITH customer_type AS (...) SELECT customer_segment, COUNT(*) AS `Number of Customers` FROM customer_type GROUP
BY customer_segment;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Top 3 products per category

```
WITH item_counts AS (...) SELECT item_rank, category, item_purchased, total_orders FROM item_counts WHERE
item_rank <= 3;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Repeat buyers by subscription

```
SELECT subscription_status, COUNT(customer_id) AS repeat_buyers FROM customer WHERE previous_purchases > 5 GROUP
BY subscription_status;
```

SQL console output: Not included in the uploaded .sql file; no values have been invented.

Revenue by age group

```
SELECT age_group, SUM(purchase_amount) AS total_revenue FROM customer GROUP BY age_group ORDER BY total_revenue
DESC;
```

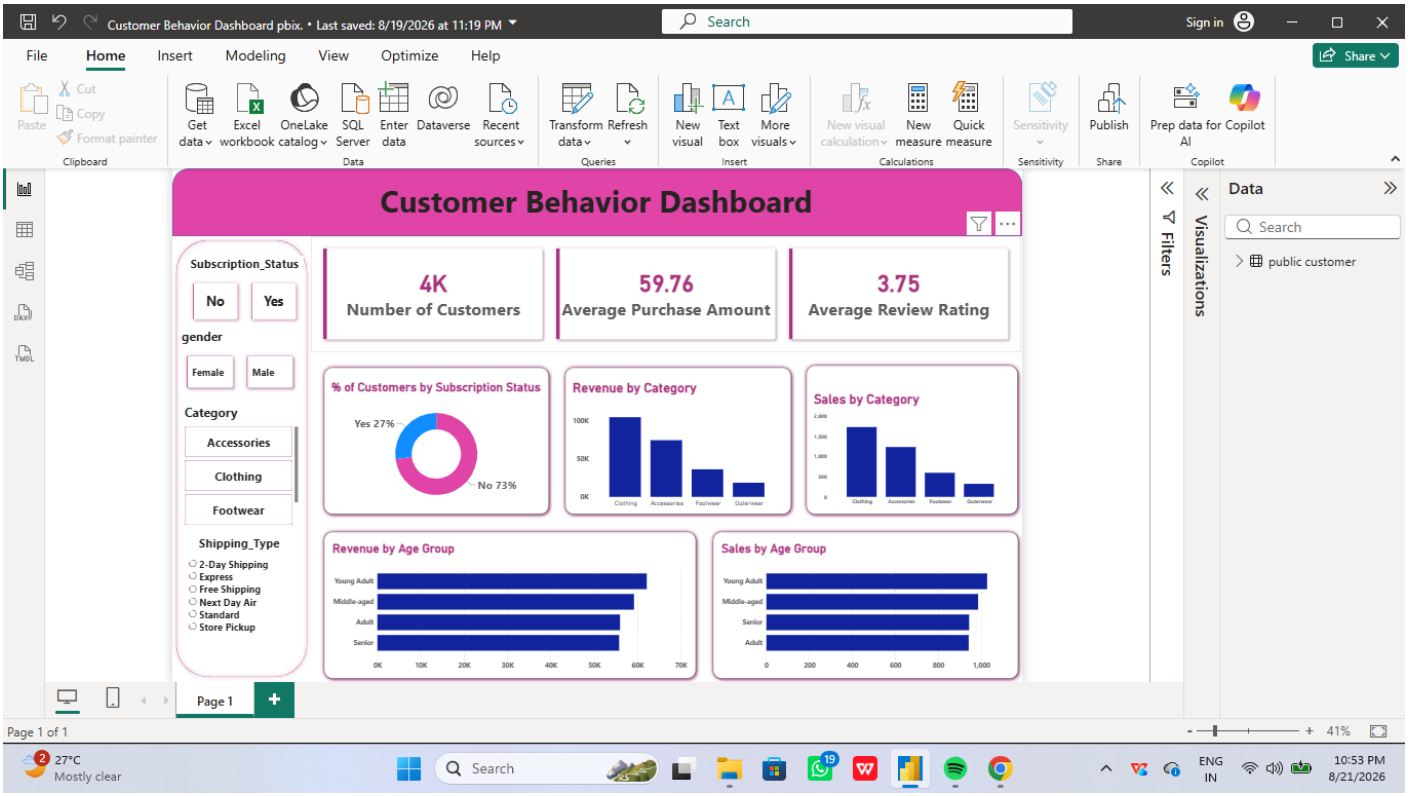
SQL console output: Not included in the uploaded .sql file; no values have been invented.

Power BI result evidence from the supplied dashboard

Dashboard evidence	Visible result
Customer KPI	4K
Average Purchase Amount	59.76
Average Review Rating	3.75
Subscription mix	Yes 27% / No 73%
Category visuals	Clothing highest; Accessories next; Footwear and Outerwear lower
Age-group visuals	Revenue and sales shown for Young Adult, Middle-aged, Adult and Senior

8. Power BI — Final Dashboard Screenshot

The screenshot below is the current Power BI dashboard supplied with the project. It contains slicers for subscription status, gender, category and shipping type, three KPI cards, a subscription donut chart, category revenue/sales charts, and age-group revenue/sales charts.



Dashboard evidence: 4K customers, 59.76 average purchase amount, 3.75 average review rating, 27% subscribed vs 73% not subscribed. The visible category charts place Clothing highest.

9. Business Insights & Recommendations

Insights supported by the uploaded notebook and dashboard evidence:

• The dataset contains 3,900 customer purchase records and the cleaned dataset has no remaining null values.

• Average purchase amount is approximately 59.76 and average review rating is approximately 3.75.

• The Power BI dashboard shows approximately 4K customers and a subscription mix of 27% Yes versus 73% No.

• Clothing is visibly the strongest category in the dashboard for both revenue and sales.

• Age-group charts allow performance comparison across Young Adult, Middle-aged, Adult and Senior customers.

Recommendations

• Target the large non-subscriber segment with conversion and retention campaigns.

• Use customer segments based on previous purchases to differentiate campaigns for new, returning and loyal customers.

• Monitor review ratings together with sales to identify products that need customer-experience improvement.

• Use category and age-group performance to guide inventory, promotions and customer targeting.

10. Conclusion

This project demonstrates an end-to-end analytics pipeline: raw customer data is prepared in Python, loaded into MySQL, analyzed with SQL, and presented through Power BI. The updated report now includes the actual Python notebook outputs and the actual dashboard screenshot. SQL queries are documented from the uploaded SQL file, while SQL-console result values are intentionally not fabricated because those results were not included in the supplied SQL file.

Final portfolio flow: CSV → Python/Pandas → MySQL → SQL Analysis → Power BI → Business Insights